Strings - indexing, slicing, and methods — review

CODE EXAMPLE

Length, indexing, and slicing

Predict before running. Which expression returns a character, which returns a substring, and which returns a number?

Code

```
1 text = "Hello World"
2 print(len(text))
3 print(text[2])
4 print(text[2:5])
```

Printed output

CODE EXAMPLE

Length, indexing, and slicing

Predict before running. Which expression returns a character, which returns a substring, and which returns a number?

Code

```
1 text = "Hello World"
2 print(len(text))
3 print(text[2])
4 print(text[2:5])
```

Printed output

```
11
l
llo
```

Exercise 0 — Length, indexing, and slicing

Write your predictions, then run the example. Write a slice that selects `"World"`. Explain why its stop is excluded. Try a negative start too.

```
1 text = "Hello World"
2 # YOUR CODE HERE: select World in two ways.
```

Exercise 0 — Length, indexing, and slicing — answer

Length is 11. Index 2 selects `"l"`. The slice selects `"llo"` at indices 2, 3, and 4. To select World:

```
1 text = "Hello World"
2 print(text[6:11])
3 print(text[-5:])
```

CODE EXAMPLE

Methods return new strings

Predict both results. Does the original string change?

Code

```
1 text = "Hello World"
2 changed = text.upper().replace("L", "S")
3 print(changed)
4 print(text)
```

Printed output

CODE EXAMPLE

Methods return new strings

Predict both results. Does the original string change?

Code

```
1 text = "Hello World"
2 changed = text.upper().replace("L", "S")
3 print(changed)
4 print(text)
```

Printed output

```
HESSO WORSD
Hello World
```

EXERCISE

Exercise 1 — Methods return new strings

Transform `"NoIce"` into `"Noice"` by chaining methods. Print the original afterward and explain the word immutable.

```
original = "NoIce"
# YOUR CODE HERE: save a changed value, then print both values.
```

Exercise 1 — Methods return new strings — answer

The shared example prints `"HESSO WORSD"`, then `"Hello World"`. After uppercasing, replacement must search for uppercase L.

```
1 original = "NoIce"
2 changed = original.lower().capitalize()
3 print(changed)
4 print(original)
```

CODE EXAMPLE

Delimiters and the mystery phrase

Recall the original phrase challenge. What is the delimiter, and what type of result does `split` return?

Code

```
1 phrase = "N O I C E"
2 pieces = phrase.split(" ")
3 print(pieces)
```

Printed output

CODE EXAMPLE

Delimiters and the mystery phrase

Recall the original phrase challenge. What is the delimiter, and what type of result does `split` return?

Code

```
1 phrase = "N O I C E"
2 pieces = phrase.split(" ")
3 print(pieces)
```

Printed output

```
['N', 'O', 'I', 'C',
'E']
```

Exercise 2 — Delimiters and the mystery phrase

Join the pieces with commas, then create lowercase `"noice"` without spaces. Explain the difference between the two `join` delimiters.

```
1 phrase = "N O I C E"
2 pieces = phrase.split(" ")
3 # YOUR CODE HERE: join with commas, then without spaces.
```

Exercise 2 — Delimiters and the mystery phrase — answer

The `split` delimiter is a space. The result is a list of five one-character strings. A comma inserts separators, while an empty string inserts nothing.

```
1 phrase = "N O I C E"
2 pieces = phrase.split(" ")
3 commas = ",".join(pieces)
4 noice = "".join(pieces).lower()
5 print(commas)
6 print(noice)
```